Functions

- A function is a reusable block of JavaScript code designed to perform a particular task.
- Instead of writing the same statements repeatedly, we can place them inside a function and call the function whenever required.
- A function executes when it is called or invoked.

Why Functions Are Used

- To avoid repeating the same code.
- To divide a large program into smaller tasks.
- To improve code readability.
- To make code easier to test and maintain.
- To reuse the same logic with different values.

Function Syntax

```
function functionName(parameters) {
   // statements
}
```

function is the keyword used to declare a function. functionName is the name given to the function. parameters are optional inputs received by the function.

Function Declaration

A function declaration defines a function using the function keyword.

```
function greet() {
   console.log("Hello Students");
}
```

The above code only defines the function. The statements inside it execute when the function is called.

Calling a Function

Calling a function means executing the function.

```
function greet() {
   console.log("Hello Students");
}

greet();
```

Output: Hello Students

Type of Functions

1. Function Without Parameters

A function can perform a task without receiving any input values.

```
function welcome() {
   console.log("Welcome to JavaScript");
}

welcome();
```

2. Function With Parameters

Parameters are variables written in the function definition. They receive values when the function is called.

```
function greet(name) {
   console.log("Hello " + name);
}

greet("ABC");
```

Here, name is the parameter and "ABC" is the argument passed to the function.

Parameters and Arguments

Term	Meaning	Example
Parameter	Variable defined in the function declaration.	name in function greet(name)
Argument	Actual value passed when calling the function.	"ABC" in greet("ABC")

Function With Multiple Parameters

```
function add(a, b) {
   console.log(a + b);
}


add(10, 20);
```

Output: 30

Return Statement

The return statement sends a value from the function back to the place where the function was called.

```
function add(a, b) {
   return a + b;
}


let result = add(10, 20);
console.log(result);
```

Output: 30

- return ends the execution of the function.
- The returned value can be stored in a variable or used in another expression.
- A function does not need to use return if it only performs an action.

Function With Return Value

```
function square(number) {
   return number * number;
}

let result = square(5);
console.log(result);
```

Output: 25

Function Without Return Value

A function can perform an action without returning a value.

```
function displayMessage() {
   console.log("Welcome");
}

displayMessage();
```

Passing Different Values

The same function can be reused with different arguments.

```
function greet(name) {
   console.log("Hello " + name);
}

greet("ABC");
greet("DEF");
greet("GHI");
```

The function is written once but can be called multiple times with different values.

Local Variables in a Function

A variable declared inside a function is normally available only within that function.

```
function showAge() {
   let age = 20;
   console.log(age);
}

showAge();
```

- The variable age is local to showAge().
- It should not be directly accessed outside the function.

Default Parameters

A default parameter provides a value when an argument is not supplied. This feature is commonly used with modern JavaScript and is also covered under ES6 Features.

```
function greet(name = "Guest") {
   console.log("Hello " + name);
}

greet();
greet("ABC");
```

Output:
Hello Guest
Hello ABC

Function Expression / Lambda Function

A function can also be stored in a variable. This is called a function expression.

```
const add = function(a, b) {
   return a + b;
};

console.log(add(10, 20));
```

Output: 30

- The function is assigned to the variable add.
- The function can be called using add().